

MobileExplorer: Accelerating On-Device Multi-Step Inference for Mobile GUI Agents via Online Exploration

ABSTRACT

Mobile graphical user interface (GUI) agents enable AI models to autonomously operate smartphones on behalf of users. However, most existing systems primarily optimize task accuracy on relatively simple benchmark tasks and rely on cloud-hosted model inference that introduces privacy concerns and network-dependent latency. As a result, fully on-device deployment of mobile GUI agents for complex, multi-step tasks remains largely underexplored. We propose MobileExplorer, a new framework that accelerates on-device multi-step inference in mobile GUI agents through online exploration. The key idea is to exploit the substantial reasoning latency of large models (LLMs/VLMs) to perform lightweight, parallel exploration of semantically relevant UI elements. Specifically, during model inference, MobileExplorer employs a relevance-driven hybrid search strategy that dynamically adjusts breadth- and depth-oriented exploration according to task intent and UI complexity, enabling efficient navigation of the UI state-action space under bounded reasoning budgets across heterogeneous tasks and UI states. To prevent speculative exploration from perturbing the primary reasoning trajectory, MobileExplorer employs a *dual-path state recovery* mechanism that restores and verifies the UI state after each exploratory action. It first performs fast, depth-bounded backtracking through reversible transitions, and falls back to deterministic action trajectory replay using a minimally logged action prefix when backtracking fails. The resulting exploration traces are then distilled into structured, state-aligned evidence, and only high-certainty signals are injected into the model prompt as structured grounding to improve subsequent reasoning accuracy and efficiency. We evaluate MobileExplorer on multiple on-device platforms, including two smartphones, an NVIDIA AGX Orin device, and a MacBook Air M4 laptop, using the commonly used AndroidWorld benchmark as well as newly designed complex tasks with dense UI elements and dynamic environments. MobileExplorer reduces the average number of reasoning steps and end-to-end latency by 23%, while maintaining or improving task success rates by up to 5%¹. A video demonstration of MobileExplorer’s performance in real-world scenarios is available at <https://youtu.be/vAGEl6-xB6Y>.

1 INTRODUCTION

Mobile graphical user interface (GUI) agents enable automatic task execution on behalf of users, such as trip planning, cross-screen record editing, or system configuration. Recent advances in large language and vision-language models (LLMs/VLMs) [1, 2, 6, 16] have enabled end-to-end mobile task automation, integrating perception, planning, and action selection within a single model [19]. These agents follow a closed-loop execution pattern: observing the UI state, reasoning about the next action, executing the action, and adapting to the resulting state.

Despite these advances, most existing mobile GUI agent systems rely on cloud-hosted models for reasoning while executing actions locally, requiring UI data transmission to remote servers. Such architecture introduces privacy risks and network-dependent latency. Moreover, prior work largely focuses on simple benchmark tasks (e.g., audio recording or note-taking) with limited interaction complexity and stable UI states. In contrast, real-world tasks involve dynamic interfaces, long-horizon navigation, and interrupt-driven events, requiring sustained multi-step reasoning and incurring significant cumulative inference latency. This motivates the development of on-device GUI agents capable of closed-loop execution for complex, multi-step tasks, where models must continuously reason over evolving UI states.

However, building a fully on-device mobile GUI agent for multi-step execution remains challenging. First, real-world mobile tasks require repeated model invocations, and each step can incur nontrivial reasoning latency. This issue is further exacerbated on mobile and edge devices, as large-model inference (LLMs/VLMs) suffers from substantial latency due to limited computation, memory, and energy budgets. For example, the VLM-based agent model MAI-UI-2B [27] incurs about 40 seconds of latency on a Samsung Galaxy S24. Second, mobile tasks exhibit highly heterogeneous UI states across applications, layouts, themes, resolutions, and device conditions. Task-relevant information is often distributed across multiple UI branches and may not be immediately visible, requiring the agent to actively reason about navigation paths rather than relying solely on the current screen. Third, mobile interfaces are inherently dynamic, with frequent changes caused by pop-ups, notification banners, soft keyboards, asynchronous content loading, and device-specific variations. Such dynamics can invalidate prior observations and often necessitate multiple rounds of reasoning and interaction to maintain task progress.

Existing approaches for improving mobile GUI agents still face significant limitations under on-device multi-step execution. Some approaches aim to reduce the number of model invocations by generating action sequences in advance [19, 20]; however, such plans are inherently brittle and often fail in the presence of dynamic UI changes. Verifier-based systems [4] shift action generation toward lightweight validation over candidate actions, but their effectiveness depends heavily on the quality of the candidate set and still incurs nontrivial reasoning overhead. Input-pruning techniques [12] aim to reduce input complexity, yet aggressive pruning risks may discard task-relevant UI elements necessary for accurate decision-making. More importantly, most existing systems adopt a strictly sequential execution paradigm that tightly couples model reasoning with GUI interaction: the device must wait for the model to complete reasoning before executing the next action. As a result, the often lengthy reasoning phase is treated as unavoidable idle time, rather than being leveraged to proactively gather or refine task-relevant UI information.

¹The source code for MobileExplorer will be released once the paper is accepted.

In this paper, we propose MobileExplorer, a new on-device mobile GUI agent framework that exploits the long reasoning latency of models (LLMs/VLMs) to perform lightweight, parallel exploration of semantically relevant UI elements. We observe that on-device mobile GUI agents exhibit a clear latency imbalance: UI perception and lightweight interactions are relatively fast, while model reasoning incurs substantial latency. Instead of remaining idle during model inference, MobileExplorer proactively explores task-relevant UI states within the model reasoning window and records the resulting interaction traces as structured exploration memory.

Specifically, to enable effective exploration under tight model reasoning time constraints, MobileExplorer adopts a relevance-driven hybrid search strategy that dynamically adjusts breadth- and depth-oriented exploration according to task intent and UI complexity, enabling efficient navigation of the UI state-action space. The resulting transitions are stored as structured exploration memory capturing both state changes and their semantic context. To prevent speculative exploration from disrupting the main model reasoning trajectory, MobileExplorer employs a *dual-path state recovery* mechanism that restores and verifies the UI state after each exploratory action. It first attempts a fast, depth-bounded backtracking path that leverages reversible UI transitions for efficient state restoration, and switches a fallback trajectory-replay path that deterministically re-executes a minimal action prefix derived from logged interaction traces when backtracking fails. Finally, MobileExplorer incorporates certainty-aware evidence distillation, which converts exploration traces into compact, state-aligned evidence for subsequent reasoning. It ranks results by state consistency, evidence gain, confidence, and safety, injecting only high-certainty signals into the model prompt as structured grounding signals to improve subsequent reasoning accuracy and efficiency. By externalizing and reusing exploration knowledge, MobileExplorer improves decision quality, reduces unnecessary trial-and-error interactions and reasoning steps, and ultimately reduces end-to-end latency.

We evaluate MobileExplorer under both benchmark and real-world deployment settings. For benchmark evaluation, we use AndroidWorld [14], a widely adopted closed-loop mobile GUI benchmark that requires agents to execute multi-step tasks over dynamic Android interfaces. For real-world evaluation, we design more complex tasks that feature dense UI layouts, pop-up interruptions, and dynamic resource conditions. Experiments are conducted across representative devices, including a Samsung Galaxy S24 smartphone, an NVIDIA AGX Orin edge device, and a MacBook Air M4 laptop. The results show that MobileExplorer reduces the average number of reasoning steps and end-to-end latency by approximately 23%, while maintaining or improving task success rates by up to 5%.

The main contributions of this work are:

- We conduct an in-depth analysis of end-to-end execution latency of on-device mobile GUI agents, and identify that model reasoning latency accumulates across multi-step execution while remaining underutilized in existing sequential interaction pipelines.
- We propose MobileExplorer, a new on-device mobile GUI agent framework that exploits model reasoning latency to perform lightweight, online exploration of task-relevant UI elements, allowing the system to gather exploration traces

as structured memory for enhancing subsequent reasoning steps.

- To enable effective and reliable exploration during model reasoning, we design three tightly coupled modules: a relevance-driven hybrid search strategy that adaptively perform breadth and depth-based exploration to probe semantically meaningful UI branches, a dual-path state recovery mechanism that restores the initial UI state when naive backtracking fails, and certainty-aware evidence distillation that converts exploration traces into compact, state-aligned prompt evidence to improve subsequent reasoning performance.
- We evaluate MobileExplorer using the AndroidWorld benchmark and newly designed real-world mobile tasks across representative devices, including Samsung Galaxy S24, NVIDIA Jetson AGX Orin, and MacBook Air M4. The results show that MobileExplorer reduces both reasoning steps and end-to-end latency by 23%, while maintaining and even improving task success rates by up to 5%.

2 RELATED WORK

Mobile GUI agent systems. Mobile GUI agents have been studied with both text-based and vision-based inputs. Text-based agents, such as AutoDroid [19], AutoDroid-V2 [20], and V-Droid [4], use accessibility trees, action histories, or candidate UI elements for planning and verification, but they miss visual cues such as icons, layouts, and spatial relations. Vision-based agents, including CogAgent [8], Mobile-Agent-V3 [24], MAI-UI [27], and STEP-UI [22], improves model performance by reasoning over screenshots. Recent work also studies long-horizon and realistic mobile settings, such as AgentProg [17] for program-structured GUI automation and AgentHazard [13] for third-party content threats. However, most existing systems still rely on cloud-based model inference and remote device control via tools like ADB [7], leading to privacy risks and network-dependent latency.

On-device deployment of mobile GUI agents. A few studies explore running language-model-based agents directly on mobile devices [19, 20]. For example, AutoDroid [19] performs local reasoning by representing GUI states as structured text and constructing UI transition graphs, while AutoDroid-V2 [20] reduces latency by generating action plans from app documentation. Concurrent systems work also optimizes the execution pipeline of on-device agents. Agent-X [3] accelerates on-device AI agents by exploiting agent-specific prompt structure for prefix caching and using LLM-free speculative decoding to reduce prefill and decode latency. These approaches are complementary to our work: they optimize model execution or text-based planning, while MobileExplorer focuses on vision-based mobile GUI agents and exploits the long VLM reasoning window to perform task-relevant online exploration in parallel. However, existing on-device GUI-agent methods rely mainly on textual input and planning priors, which cannot be directly applied to vision-based agents that must reason over raw screenshots. Moreover, vision-based agents require VLM inference over high-dimensional visual inputs, introducing substantially higher computational overhead for on-device deployment.

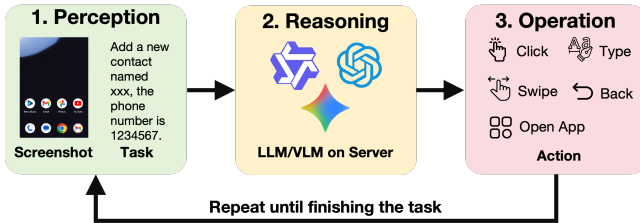


Figure 1: End-to-end workflow of a mobile GUI agent. The agent observes the current UI state, performs model-based reasoning with LLMs/VLMs, and outputs actions that are mapped to executable GUI operations.

Offline knowledge base construction for mobile GUI agents.

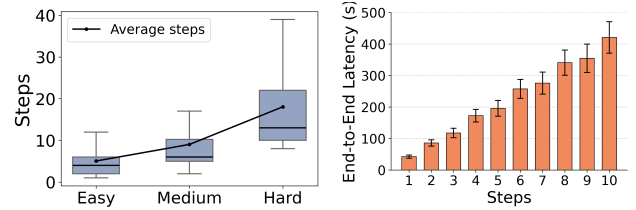
Some mobile GUI agents improve decision making by incorporating prior knowledge from explored interfaces. For example, AutoDroid [19] constructs a UI transition graph as structured memory, while AutoDroid-V2 [20] leverages app documentation and generated task samples for planning. Other works build such knowledge through offline exploration. GUI-explorer [21] mines transition-aware knowledge from state-action traces, and LLM-Explorer [26] constructs reusable repositories of UI states and interaction graphs via large-scale app exploration. However, these approaches depend on offline-collected trajectories or pre-built knowledge bases, which are costly to construct and difficult to generalize to dynamic interfaces. In contrast, our work performs lightweight online exploration during inference without requiring offline knowledge construction, enabling adaptation to dynamic real-world mobile applications.

3 A MOTIVATION STUDY

3.1 Workflow of Mobile GUI Agents

Figure 1 illustrates the typical end-to-end workflow of a mobile GUI agent, which consists of perception, reasoning, and operation. (1) *Perception*. Given the current screen, the agent first captures a screenshot that reflects the visual layout, icons, text, and spatial relationships among UI elements. (2) *Reasoning and planning*. For text-based agents, the system converts the UI state into structured textual representations, such as accessibility-tree nodes, element labels and control types and feeds them together with the task instruction into an LLM for reasoning. For vision-based agent, an image encoder converts the screenshot into visual embeddings. These embeddings are then combined with textual tokens from user instructions, task descriptions, or dialogue context, and fed into a VLM for reasoning. The LLM or VLM then outputs executable GUI actions, such as tapping icons, typing text, scrolling, or navigating across apps. (3) *Operation*. The system executes these actions on the device and repeats the perception-reasoning-operation loop until the task is completed. For example, to accomplish the task “Turn on Wi-Fi”, the agent may need to open the Settings app, scroll to locate the Wi-Fi menu, and tap the corresponding switch.

Different from static GUI datasets that evaluate a single predicted action from a recorded screen, a deployed mobile GUI agent must operate a live device in a closed loop. Each action changes the screen state, and later decisions depend on the correctness and latency of previous actions. This closed-loop pattern is shared by both accessibility-tree-based text models and screenshot-based vision-language models. As a result, real mobile GUI automation is



(a) Required steps across task difficulty. (b) End-to-end latency vs. reasoning steps.

Figure 2: Impact of reasoning steps in on-device mobile GUI agent systems.

inherently *multi-step*, especially for tasks that involve searching, switching pages, handling pop-ups, or navigating dense third-party applications.

3.2 Impact of Reasoning Steps

To understand the impact of reasoning steps on on-device mobile GUI agents, we measure both the number of steps required by different tasks and the resulting end-to-end latency. We conduct a preliminary study using AndroidWorld [14], a closed-loop mobile GUI benchmark in which agents interact with live Android applications. Experiments are performed on a Samsung Galaxy S24 equipped with 12GB RAM, with the 2B vision-language model MAI-UI [27] deployed locally through llama.cpp without any cloud offloading.

We first analyze the number of reasoning steps required across tasks of varying difficulty. Following AndroidWorld’s task organization, tasks are grouped into easy, medium, and hard categories based on interaction complexity, including whether target UI elements are directly visible, whether multi-screen navigation is required, and whether the agent must perform search, editing, or intermediate-state reasoning. As shown in Fig. 2a, harder tasks generally require more reasoning steps and exhibit substantially larger variance. Easy tasks typically involve short and deterministic interaction trajectories, whereas medium and hard tasks often require navigating multiple pages, locating hidden UI elements, or resolving ambiguous interaction branches.

We then use the task of *recording and saving an audio file* as a representative case study to analyze how the number of reasoning steps affects end-to-end system latency. Fig. 2b shows that end-to-end latency increases almost monotonically with the number of reasoning steps. This trend arises because each additional step introduces another model inference call, while on-device reasoning dominates the per-step execution cost. Therefore, reducing unnecessary reasoning steps is critical for lowering overall latency in complex mobile GUI tasks.

These observations suggest that the inefficiency of on-device GUI agents arises not merely from slow individual inference calls, but from the combined effect of long-horizon decision making and high per-step reasoning overhead. This limitation is also consistent with prior on-device agent systems. For example, AutoDroid-V2 [20] reports an average on-device LLM inference latency of 46.3 seconds per task on a Snapdragon 8 Gen 2 platform, while recent systems work such as Agent-X identifies model inference as a major bottleneck in end-to-end on-device agent pipelines [3]. In our

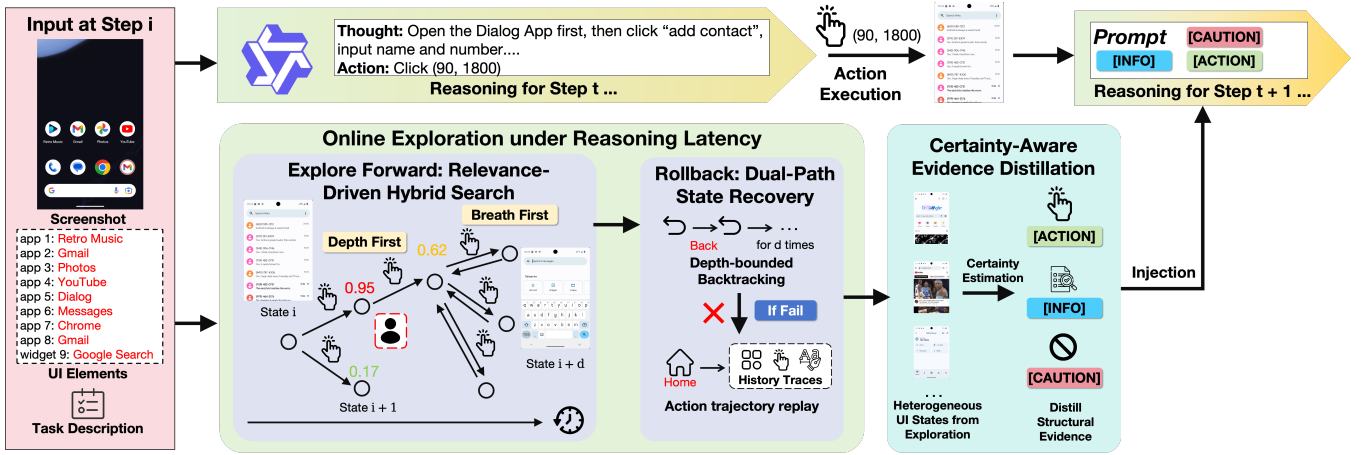


Figure 3: System overview of MobileExplorer. During each reasoning step, MobileExplorer performs relevance-driven hybrid search in parallel with model reasoning, dynamically adjusting exploration breadth and depth using the reasoning prior and session-local exploration memory. It then restores the UI state through dual-path state recovery and injects high-confidence, state-aligned evidence into subsequent reasoning steps.

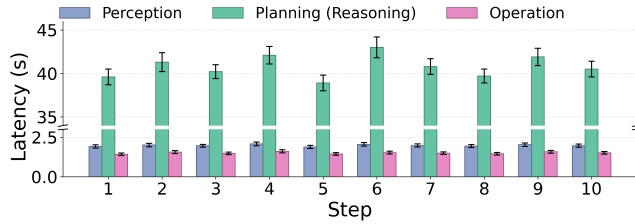


Figure 4: Latency breakdown across execution steps. Model reasoning dominates per-step latency, leaving substantial idle time for lightweight UI exploration.

measurements, even a compact 2B model running fully on a Samsung Galaxy S24 still incurs more than 20 seconds of reasoning latency per decision step.

3.3 Opportunity for Online Exploration

We further analyze the latency of each component in the end-to-end workflow in Fig. 1, including perception, reasoning, and UI operation. Fig. 4 presents the latency breakdown across execution steps for the AndroidWorld task *recording and saving an audio file*. The results show that each step incurs substantial model reasoning latency, whereas perception and UI operations contribute only a small fraction of the total execution time. This finding indicates that the dominant portion of system latency arises from waiting for the model to generate the next action before interacting with the UI.

This latency imbalance reveals a key opportunity for accelerating on-device mobile GUI agents. Each model reasoning step typically requires tens of seconds, while lightweight UI interactions—such as taps, swipes, and screen transitions—usually complete within only 1–2 seconds. However, existing agents generally adopt a strictly sequential execution pipeline: the system remains idle while waiting for reasoning to complete before performing the next interaction.

We argue that this otherwise idle interval can instead be exploited through online exploration. By proactively probing selected

UI elements and observing the resulting screen transitions during model inference, the agent can acquire additional UI context that reduces uncertainty in subsequent decisions and shortens the overall interaction trajectory. This opportunity is particularly valuable for complex multi-step tasks. While simple deterministic tasks may require little exploration, tasks involving dense UI layouts, hidden interaction branches, or multi-page navigation can substantially benefit from contextual information that would otherwise require additional trial-and-error interactions after reasoning completes.

3.4 Summary

We summarize the key findings from our motivation study as follows:

- On-device mobile GUI agents suffer from accumulated latency during multi-step execution, as each interaction step introduces an additional costly model reasoning call.
- UI interactions are significantly faster than model reasoning, creating otherwise idle intervals during inference that can be exploited for lightweight online exploration.
- Online exploration is particularly beneficial for complex tasks with high UI uncertainty and long interaction horizons, where reducing unnecessary reasoning steps can directly lower end-to-end latency.

4 SYSTEM OVERVIEW

In this paper, we propose MobileExplorer, a new on-device mobile GUI agent framework that exploits the long reasoning latency of models (LLMs/VLMs) to perform lightweight, parallel exploration of semantically relevant UI elements. As illustrated in Fig. 3, unlike conventional GUI agents that follow a sequential perception–reasoning–operation pipeline and remain idle during model inference, MobileExplorer proactively explores task-relevant UI states within the model reasoning window and records the resulting interaction traces as structured exploration memory.

Specifically, to enable effective exploration under tight model reasoning time constraints, MobileExplorer adopts a relevance-driven hybrid search strategy that dynamically adjusts breadth- and depth-oriented exploration according to task intent and UI complexity, enabling efficient navigation of the UI state-action space. The resulting transitions are stored as structured exploration memory capturing both state changes and their semantic context. To prevent speculative exploration from disrupting the main model reasoning trajectory, MobileExplorer employs a *dual-path state recovery* mechanism that restores and verifies the UI state after each exploratory action. It first attempts a fast, depth-bounded backtracking path that leverages reversible UI transitions for efficient state restoration, and switches a fallback trajectory-replay path that deterministically re-executes a minimal action prefix derived from logged interaction traces when backtracking fails. Finally, MobileExplorer incorporates certainty-aware evidence distillation, which converts exploration traces into compact, state-aligned evidence for subsequent reasoning. It ranks results by state consistency, evidence gain, confidence, and safety, injecting only high-certainty signals into the model prompt as structured grounding signals to improve subsequent reasoning accuracy and efficiency.

Overall, MobileExplorer establishes a parallel execution pipeline in which the model reasons about the current step while online exploration simultaneously gathers contextual knowledge for future interactions. By exploiting otherwise idle reasoning time, MobileExplorer externalizes and reuses exploration knowledge to improve decision quality, reduce unnecessary trial-and-error interactions and redundant reasoning steps, and ultimately lower end-to-end latency of on-device mobile GUI agent systems.

5 DESIGN OF MOBILEEXPLORER

5.1 Problem Formulation

We consider an on-device mobile GUI agent that interacts with an application to finish a user-defined task through multi-step interactions, including perception, reasoning and operations. At step i , the agent observes the current UI state s_i through a screenshot, an accessibility tree, or both and receives a task description $\mathcal{G}$. Due to the partial observability of mobile interfaces, the full UI structure cannot be known a priori and can only be revealed through interaction.

State and Action Space. At step i , the agent observes the current UI state s_i and parses the set of interactive UI elements:

$$\mathcal{A}_i = a_i^{(1)}, a_i^{(2)}, \dots, a_i^{(K_i)}. \quad (1)$$

Each action corresponds to interacting with a UI element (e.g., click or scroll), which may trigger a transition to a new UI state.

Latency-Constrained Interactions. On-device model inference incurs significant latency. Let τ_i^{infer} denote the reasoning latency at step i . During this period, the device remains idle from the interaction perspective, providing an opportunity to perform lightweight online exploration without increasing user-perceived delay when exploration and recovery fit within the reasoning window.

System Objective. The objective of MobileExplorer is to maximize the probability of completing a task while minimizing end-to-end

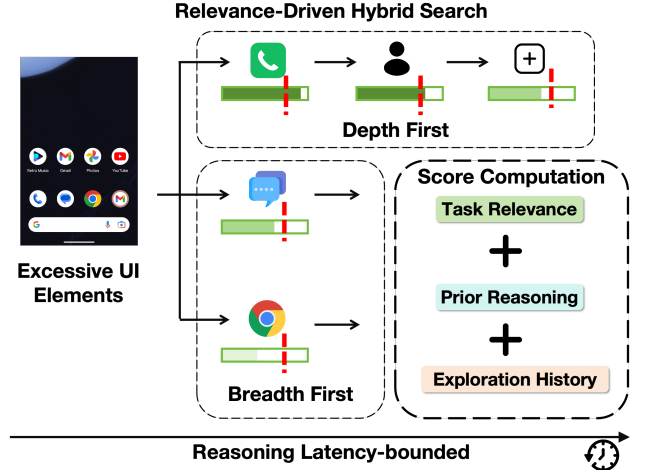


Figure 5: Relevance-driven hybrid search. During each model reasoning step, the system uses the previous reasoning prior and session-local exploration memory to select safe and task-relevant UI branches, then adaptively adjusts exploration breadth and depth within model reasoning time.

latency under on-device latency constraints:

$$\max_{\pi} \Pr(s_N \in \mathcal{S}_{\text{goal}}(\mathcal{G})), \quad \text{s.t.} \quad \sum_i (\tau_i^{\text{infer}} + \tau_i^{\text{interact}}) \leq T, \quad (2)$$

where T is the latency budget and π denotes the interaction policy.

To achieve this objective, MobileExplorer leverages the otherwise idle reasoning latency to perform lightweight online exploration and gather additional UI context without increasing the overall execution delay. This design introduces several challenges, including selecting informative interactions under strict time constraints, reliably restoring the original UI state after speculative exploration, and distilling discovered UI states into compact, high-certainty evidence for subsequent reasoning.

5.2 Explore Forward: Relevance-Driven Hybrid Search

As motivated by Section 3, the goal of online exploration is to proactively gather additional UI context during the model inference window to improve subsequent reasoning. We refer to this module as *relevance-driven hybrid search*: during model reasoning, the agent performs short speculative actions to uncover hidden UI branches and collect auxiliary information that may not be visible from the current screen alone. The explored information is reused only after state recovery, state alignment, and evidence filtering.

Challenges. First, online exploration must operate strictly within the latency of model inference at each reasoning step, so it is bounded by both branch count and speculative depth. Second, a typical screen contains many clickable candidates, but only a small subset can reduce uncertainty for the current task. Third, the usefulness of a branch is not fixed across apps and tasks: a search box may be useful for an information lookup task, while the same type of control may be irrelevant in a form-editing task. Therefore, exploration should not rely on a fixed hand-crafted score. Instead, it should adapt to the current reasoning prior, observed branch

outcomes, missing task information, latency budget, and recovery risk.

5.2.1 Hybrid Search Strategy for Exploration. To efficiently identify exploration candidates, the probing module parses the accessibility tree of the current screen s_i and extracts the clickable set:

$$\mathcal{A}_i = a_i^{(1)}, a_i^{(2)}, \dots, a_i^{(K_i)}. \quad (3)$$

Each candidate is represented as lightweight text (e.g., label, content, identifier) plus action operator, layout region, and semantic role. A simple relevance-only search is insufficient because many useful UI controls have short or generic labels, such as ‘Search’, ‘More’, or ‘Next’. Their usefulness depends not only on semantic similarity to the task, but also on the previous reasoning output, the current exploration history, and whether the branch can be safely explored and recovered. Therefore, MobileExplorer formulates online exploration as a dynamic node-weighted search over the local UI state-action space.

For each candidate action $a_i^{(k)}$, MobileExplorer maintains a node weight $w_i(a_i^{(k)})$ that estimates how useful this candidate is for the current reasoning step. The node weight is computed from two sources of lightweight context: the reasoning prior extracted from the previous model output, and a session-local exploration memory accumulated within the current app-task session.

Reasoning Prior. In addition to the task description, MobileExplorer uses a lightweight reasoning prior extracted from the previous model output. After the model finishes step $i - 1$, the system parses its reasoning text and action decision to obtain:

$$\mathcal{P}_{i-1} = K_{i-1}^{\text{target}}, A_{i-1}^{\text{expect}}, K_{i-1}^{\text{risk}}, U_{i-1}^{\text{miss}}, \quad (4)$$

where K_{i-1}^{target} denotes target keywords mentioned by the model, such as names, files, cities, contacts, or settings; A_{i-1}^{expect} denotes expected UI affordances such as search boxes, list entries, detail pages, or input fields; K_{i-1}^{risk} denotes risk-related keywords such as delete, submit, send, call, confirm, or pay; and U_{i-1}^{miss} denotes missing or unresolved information from the previous reasoning step. This prior is extracted through lightweight keyword and template matching from the model output and does not require an additional model invocation.

Session-local Exploration Memory. For each app-task session, MobileExplorer maintains a session-local exploration memory $\mathcal{M}_i$. The memory stores explored element signatures, resulting screen summaries, candidate-hit outcomes, slot coverage, no-effect probes, state-match results, and rollback success. Each branch signature b is associated with a utility estimate $u_i(b)$ that reflects how useful this branch has been in the current session. After each probe, the utility is updated using the observed branch outcome:

$$u_{i+1}(b) = (1 - \rho)u_i(b) + \rho \cdot \phi(o_b), \quad (5)$$

where o_b is the observation produced by branch b , ρ is an update rate, and $\phi(o_b)$ is a lightweight branch outcome value. In implementation, $\phi(o_b)$ increases when the branch hits a candidate mentioned in the reasoning prior, fills a missing task slot, reveals useful UI structure, matches the current state after alignment, or can be safely recovered. It decreases when the probe has no effect, leads to an unrelated state, repeatedly fails to provide evidence, or incurs high recovery risk.

5.2.2 Node Weight Estimation and Adaptive Search. Given task $\mathcal{G}$, reasoning prior $\mathcal{P}_{i-1}$, and exploration memory $\mathcal{M}_i$, MobileExplorer computes the node weight of each candidate using four lightweight signals.

First, semantic relevance measures whether the candidate text is related to the task:

$$r(a_i^{(k)}) = \text{sim}(e(a_i^{(k)}), e(\mathcal{G})), \quad (6)$$

where $e(\cdot)$ encodes UI and task text and sim is cosine similarity. This score captures task-level semantic similarity, but it is only one signal because useful UI elements may have short labels or weak textual overlap with the task.

Second, prior match measures whether the candidate label, operator, or semantic role overlaps with the target keywords, expected affordances, or unresolved information needs in $\mathcal{P}_{i-1}$. For example, if the previous reasoning output indicates that the agent should search for a contact name, search boxes and result-list entries receive higher prior match.

Third, branch utility comes from the session-local exploration memory and reflects whether similar branches have previously produced useful observations in the same app-task session. Branches that repeatedly produce no-effect probes or unrelated screens are down-weighted, while branches that previously revealed useful evidence are promoted.

Fourth, risk penalty suppresses unsafe speculative actions. Candidates whose labels or operators match K_{i-1}^{risk} , such as delete, save, confirm, submit, send, call, pay, or toggle, are not directly executed during speculative exploration; they are filtered out or recorded as caution observations.

The final node weight is computed as:

$$w_i(a_i^{(k)}) = \lambda_r r(a_i^{(k)}) + \lambda_p p(a_i^{(k)}, \mathcal{P} * i - 1) + \lambda_u u_i(b_k) - \lambda_h h(a_i^{(k)}), \quad (7)$$

where $p(a_i^{(k)}, \mathcal{P} * i - 1)$ denotes prior match, $u_i(b_k)$ denotes the session-local branch utility of the branch associated with candidate $a_i^{(k)}$, and $h(a_i^{(k)})$ denotes the risk penalty. The weights are fixed constants and are only used to rank candidates within the current reasoning window.

Based on the ranked node weights, MobileExplorer adaptively selects exploration roots and continuation depth. If the previous reasoning prior is confident and high-weight candidates are concentrated around a few branches, MobileExplorer uses a smaller root budget and allows deeper continuation, such as depth-2 probing. If the candidate weights are dispersed, the prior is uncertain, or the task has missing slots, MobileExplorer uses broader but shallower exploration to cover more possible branches. If the remaining reasoning-time budget is small or the estimated recovery risk is high, MobileExplorer reduces depth and avoids risky continuation. This adaptive policy implements a relevance-driven hybrid search over the local UI state-action space while keeping exploration bounded within the model reasoning window.

Given the selected candidates, MobileExplorer performs short speculative branches by tapping or scrolling the corresponding UI elements and observing the resulting screen transitions. Candidate repeats are suppressed, and observed screens are parsed again for evidence only after branch execution and recovery. Each completed

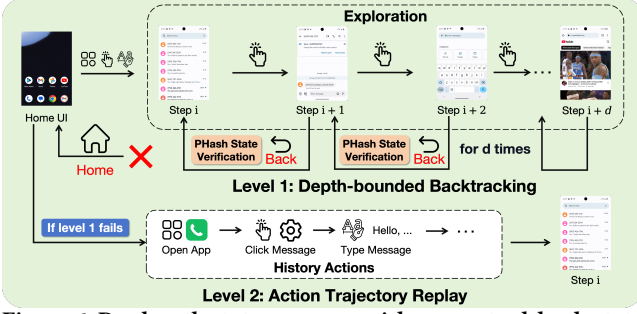


Figure 6: Dual-path state recovery with perceptual-hash state verification and deterministic trajectory replay.

probe produces an exploration observation that records the trigger element, resulting screen, salient labels, detected UI structures, transition outcome, and recovery status. These observations are not directly injected into the prompt; they are passed to the evidence distillation module for alignment, filtering, and type-aware rendering.

5.3 Rollback: Dual-Path State Recovery

In mobile on-device exploration, every speculative probe mutates the live UI. Unlike simulators, we cannot clone app state before branching, so exploration can drift the working screen if recovery is unreliable. This module therefore performs *dual-path state recovery*: a fast pHash-guided, depth-bounded backtracking path for common reversible transitions, plus a deterministic trajectory-replay fallback to restore the original context when backtracking is not safe.

Challenges State recovery is difficult for two reasons: (1) many transitions are not perfectly invertible (a back action may pop multiple layers or diverge), and (2) the same navigation path can still land on visually different screens due to asynchronous updates. At step i , only screenshot s_i is observable; the latent app state may still drift, so recovery must be robust rather than exact inversion. The one-line backtracking path is therefore insufficient when permission prompts, transient banners, or confirmation windows appear during probing; even valid back sequences can miss the exact start state. Our recovery logic tolerates minor visual variations while preserving alignment to the intended UI context before continuing reasoning.

To safely reuse exploration results, MobileExplorer must restore the UI state where exploration started before the model-selected action is executed. As shown in Fig. 6, MobileExplorer first attempts fast depth-bounded backtracking with pHash verification. This path handles the common case where exploratory transitions are reversible. If backtracking cannot restore the aligned state, MobileExplorer falls back to deterministic trajectory replay from a logged action prefix. This second path is slower but more robust to dialogs, skipped pages, and dynamic UI changes.

5.3.1 Depth-Bounded Backtracking. Exploration is performed with a bounded depth d . Let the exploration start from UI state s_i . Meanwhile, the system records the interaction trace that led to this exploration state. We compute the perceptual hash of the starting screen as

$$H_0 = H(s_i), \quad (8)$$

where $H(\cdot)$ denotes the pHash function. We use screenshot-based verification instead of accessibility-tree because capturing screenshots on mobile devices is significantly faster than retrieving the accessibility tree. Frequent screen verification is required during backtracking, and screenshot capture introduces much lower latency. Perceptual hashing further allows efficient comparison between screens while tolerating minor visual differences caused by dynamic UI elements. After finishing an exploratory branch, the agent issues the back action up to d times to return to the previous screens. After each backtracking step k , the current screen s_{r_k} is captured and verified against the starting screen:

$$d_H(H(s_{r_k}), H_0) \leq \delta, \quad (9)$$

where $d_H(\cdot)$ denotes the Hamming distance between perceptual hashes and δ is a small threshold allowing minor UI variations (e.g., scrolling offsets or dynamic content). If the condition holds, the system concludes that the original UI state has been successfully restored.

5.3.2 Deterministic Trajectory Replay. If the Level-1 backtracking fails (e.g., due to irreversible UI transitions or navigation inconsistencies), the system performs a fallback recovery. For example, during exploration an interaction may trigger a confirmation dialog or permission prompt that was not previously visible. In such cases, issuing the *back* action may dismiss the dialog and return to an earlier screen rather than the exact exploration starting state. As a result, the agent cannot reliably restore the original UI state through simple backtracking.

To recover from such situations, the system falls back to a deterministic reconstruction procedure. Let the interaction trace that led to the exploration starting state be

$$\Pi_i = (a_1, a_2, \dots, a_i), \quad (10)$$

where a_k denotes the action executed at step k . The agent first returns to a stable anchor state, such as the Home screen or app entry state, and then deterministically replays the minimal logged action prefix in Π_i to reconstruct the UI state s_i . Since exploration depth is small in practice, the replay sequence is typically short and introduces limited overhead compared to model inference latency.

After replay, the system does not blindly assume success. It verifies the recovered screen using the same activity and pHash checks. If the replayed state still does not match the recovery anchor, MobileExplorer discards the evidence from the unstable branch and starts a fresh perception-reasoning step from the current screen instead of executing a potentially stale model action.

5.4 Certainty-aware Evidence Distillation

Exploration happens during the model reasoning window, while the final action may move the agent to a different screen. Therefore, MobileExplorer does not directly append exploration traces to the prompt. Instead, it first aligns explored observations with the post-action screen, estimates their confidence, and injects only compact, state-aligned evidence into subsequent prompts.

Challenges. Exploration-derived observations face three challenges. First, they may become stale if the main reasoning action reaches a different UI state. Second, exploration may reveal many elements in a short window, and injecting all of them may add noise and

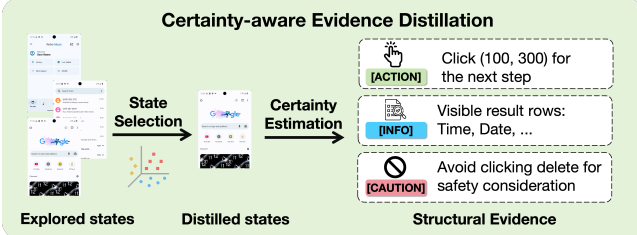


Figure 7: Certainty-aware evidence distillation and injection. The agent aligns explored observations with the current screen, ranks evidence by confidence, and injects compact, structured evidence into the next prompt.

prompt overhead. Third, different observations should be used differently: some provide contextual information, some suggest a possible next action, and others warn against unhelpful or unsafe branches. Therefore, MobileExplorer must align observations with the current state, rank their usefulness, and render them as structured evidence.

5.4.1 UI State Selection. During exploration, the agent may visit multiple intermediate UI states. Each visited screen is recorded as a compact observation containing the foreground activity, perceptual hash (pHash), salient UI labels, rollback status, transition outcome, and parsed UI elements. After the model-selected action reaches screen s_{i+1} , MobileExplorer compares these observations with the current screen and keeps only state-aligned results:

$$O_i^{\text{match}} = o_j \in O_i \mid \text{Aligned}(o_j, s_{i+1}). \quad (11)$$

Here, $\text{Aligned}(\cdot)$ checks whether the explored branch is rollback-verified and whether the explored observation matches the current screen by foreground activity and either visual similarity or salient UI-label consistency.

To tolerate dynamic UI changes, MobileExplorer complements pHash matching with anchor-label overlap. Let $L(o_j)$ be the informative labels observed in an explored state and $L(s_{i+1})$ be the informative labels on the current screen. The anchor consistency is estimated by

$$J(o_j, s_{i+1}) = \frac{|L(o_j) \cap L(s_{i+1})|}{|L(o_j) \cup L(s_{i+1})|}. \quad (12)$$

An observation is treated as aligned when the foreground activity is consistent and either the pHash distance is small or the anchor-label overlap is sufficiently high. This filters unrelated or unstable exploration results while remaining robust to notification banners, dynamic content loading, and minor visual changes.

5.4.2 Certainty-based Element Distillation. The matched observations may still contain irrelevant, redundant, or unsafe information. MobileExplorer therefore estimates a lightweight confidence score for each matched observation o_j using three signals.

Prior support measures how well the observation matches the previous reasoning prior $\mathcal{P}_{i-1}$, including target keywords, expected UI affordances, and unresolved information needs. *Observation value* measures whether the branch provides useful new information, such as an answer fact, a missing slot, a form or search schema, a concrete target, or a clear negative outcome. *Current-screen consistency* measures whether the evidence remains usable after the

main action reaches s_{i+1} , such as whether the target is still visible or shares stable anchor labels with the current screen.

These signals are combined into a lightweight confidence score:

$$C(o_j) = \alpha P(o_j, \mathcal{P}_{i-1}) + \beta V(o_j) + \gamma G(o_j, s_{i+1}) - \eta R(o_j), \quad (13)$$

where $P(o_j, \mathcal{P}_{i-1})$ denotes prior support, $V(o_j)$ denotes observation value, $G(o_j, s_{i+1})$ denotes current-screen consistency, and $R(o_j)$ denotes risk penalty. The risk penalty is high for stale, destructive, external, or commit-like observations. The coefficients are fixed constants and are used only to rank evidence candidates within a step; MobileExplorer does not train an additional confidence model. An observation is considered high-confidence when $C(o_j)$ exceeds a fixed threshold and does not violate hard safety rules.

5.4.3 Structured Evidence Injection. After confidence filtering, MobileExplorer converts selected observations into structured evidence rather than raw element dumps. We use three evidence types: *INFO_HINT*, *ACTION_HINT*, and *CAUTION_HINT*. An *INFO_HINT* contains factual or structural information, such as answer-related facts, visible values, search results, form fields, filters, or UI schemas. An *ACTION_HINT* suggests a possible next action only when the target remains visible, task-relevant, and safe on the current screen. A *CAUTION_HINT* records irrelevant, stale, unhelpful, or risky branches, such as no-effect probes, unrelated transitions, or commit-like controls.

The selected evidence forms the state-aligned exploration context C_i , which is appended to the next prompt. Each evidence item is rendered with an explicit type tag and a short natural-language description. MobileExplorer orders evidence by role: information hints first, action hints second, and caution hints last. Only the top-ranked evidence items are injected to limit prompt overhead. The prompt also states that exploration evidence is contextual: action hints should be followed only if the target remains visible and task-relevant, and the model should trust the current screen if it conflicts with injected evidence.

After the model generates the next reasoning output, MobileExplorer extracts a new reasoning prior from that output and uses it to guide the next round of exploration. Evidence that fails state matching, rollback verification, confidence filtering, safety checks, or evidence-type constraints is not injected as positive guidance; its rejection reason is recorded for debugging and future policy analysis.

6 EVALUATION

6.1 Experiment Setup

Benchmark. We evaluate MobileExplorer on AndroidWorld [14], a task-oriented mobile GUI interaction benchmark that requires multi-step reasoning over real smartphone interfaces (details in Table 1). Unlike static GUI datasets such as Android-in-the-Wild [15] and AndroidControl [11], which rely on offline interaction traces, AndroidWorld provides an interactive environment where agents operate real Android applications through system-level control interfaces. This closed-loop setting captures dynamic execution conditions such as runtime delays, UI transitions, and system responses, better reflecting practical mobile assistant scenarios where interaction efficiency and system responsiveness are critical. Table 1

Difficulty	Count	Ratio	Example of Task
Easy	61	52.6%	AudioRecorderRecordAudio
Medium	36	31.0%	NotesToDoItemCount
Hard	19	16.4%	ExpenseAddMultipleFromMarkor

Table 1: Task summary of AndroidWorld [14].

summarizes the key characteristics of the AndroidWorld benchmark. In addition to the benchmark evaluation, we also conduct an end-to-end real-world case study on a physical smartphone to validate the practicality of MobileExplorer in real deployment scenarios.

End-to-end Implementation. MobileExplorer is implemented as an end-to-end mobile GUI agent system. However, for evaluation on the commonly used AndroidWorld benchmark, the tasks are executed in an Android emulator on a host machine, which provides the interaction environment and GUI states (e.g., screenshots and accessibility trees), while the reasoning model runs only on the designated on-device platform. The agent communicates with the model through HTTP requests. Unless otherwise specified, this target platform is the Jetson AGX Orin. The models are deployed using llama.cpp with Q8 quantization and served through the vLLM framework, providing a unified API interface for the AndroidWorld agent. On the smartphone platform used in the cross-device study, llama.cpp runs inside the Termux application to enable local model execution. The evaluation is primarily based on a 4B VLM [22] and a 4B LLM [23].

Devices. We use three representative on-device platforms, including a Samsung Galaxy S24 smartphone, an NVIDIA Jetson AGX Orin, and a MacBook Air M4 laptop. To avoid ambiguity in result aggregation, we use the Jetson AGX Orin as the default inference device for all experiments unless a subsection explicitly studies different devices. The smartphone and laptop results are only used in the cross-device overhead study in Section 6.5.

Baselines. We compare MobileExplorer with the following representative baselines based on 4B VLMs/LLMs.

- **Vision-based Agent [14]:** A closed-loop mobile GUI agent that uses a VLM to reason over the current screenshot and predicts the next GUI action in a sequential perception-reasoning-operation loop.
- **Text-based Agent [14]:** A closed-loop mobile GUI agent that uses accessibility-tree observations instead of screenshots and performs step-wise action prediction with an LLM.
- **Input-pruning VLM Agent [12]:** A VLM-based GUI agent that reduces the visual input cost by pruning screenshot regions or visual tokens before model reasoning, while still following the same step-wise execution pipeline.
- **Offline Exploration Agent [21, 25, 26]:** A GUI agent that builds reusable UI knowledge from offline exploration traces or pre-constructed transition repositories, and uses the stored knowledge to guide later task execution.
- **Multi-agent System [5]:** A mobile GUI agent framework that decomposes task execution into multiple cooperating agents, such as planning, decision, and reflection agents, to track task

Method	AndroidWorld (%)
MobileGPT [10]	23.0
M3A (a11y, GPT-4-Turbo) [14]	30.6
M3A (a11y, Gemini-2.5-Pro) [14]	31.0
M3A (SoM, GPT-4-Turbo) [14]	25.4
M3A (SoM, Gemini-2.5-Pro) [14]	39.7
GLM-4.1V-9B-Thinking [9]	41.7
UI-TARS (UI-TARS-7B) [18]	33.0
MobileExplorer	50.9

Table 2: Success Rate (%) on AndroidWorld.

progress, maintain focus memory, and correct erroneous operations during execution.

- **Script-based Agent [20]:** A document-guided GUI agent that converts mobile task automation into a code generation problem. Instead of querying the model at each UI state, it generates a multi-step control script from the user task and app document, then executes the script with a domain-specific code executor.

We choose these baselines to cover the main execution styles relevant to MobileExplorer: sequential VLM/LLM reasoning, visual-input pruning, offline knowledge construction, multi-agent coordination, and script-based agent. Beyond the above baselines, we further compare MobileExplorer with existing methods reported on the AndroidWorld [15] leaderboard, while noting that leaderboard numbers may use different model backends, input modalities, and deployment settings.

Evaluation metrics. We evaluate MobileExplorer in terms of both effectiveness and system efficiency on real on-device execution.

Success rate measures the percentage of tasks successfully completed within a predefined step budget, indicating the overall task-solving capability of the agent. **Total steps** denotes the number of interaction steps required to finish a task, reflecting decision efficiency and the amount of trial-and-error during long-horizon execution. **Latency** is evaluated at two granularities. Step latency is defined as the time of a single interaction cycle, measured from capturing the current screenshot s_i to completing the execution of the selected action a_i . This includes perception, model reasoning, online exploration, and action execution. End-to-end latency is defined as the total task completion time from task initialization to the final successful state, i.e., the sum of all step latencies across the task. All aggregate results are computed at the task level. For experiments with repeated runs, we first average repeated runs of the same task and then average across tasks in the corresponding group. We also measure overhead like CPU usage, memory and power consumption in Section 6.5.

6.2 Overall Performance

We evaluate MobileExplorer on the AndroidWorld benchmark with model inference running on the default Jetson AGX Orin platform. As shown in Table 2 and Fig. 8a, MobileExplorer achieves a success rate of 50.86% (59/116 tasks), which is the best among all compared methods. Compared with the VLM-based model, which achieves

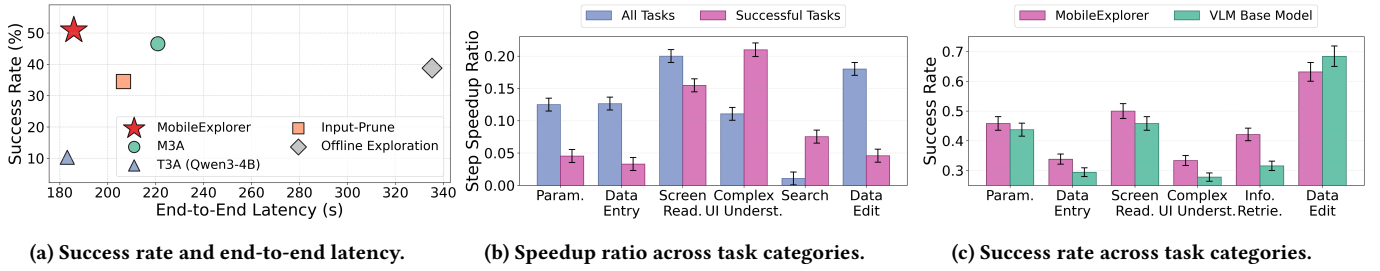


Figure 8: Overall performance on AndroidWorld.

46.55%, this corresponds to a 9.3% relative improvement. This improvement indicates that integrating latency-bounded online exploration into the reasoning process allows the agent to obtain more useful UI context and make better interaction decisions. As a result, MobileExplorer improves the ability of mobile GUI agents to complete long-horizon tasks under the on-device setting.

Meanwhile, MobileExplorer also improves the efficiency of task execution. MobileExplorer requires only 9.24 interaction steps on average, compared with 10.93 steps for VLM base model, corresponding to a 15.5% reduction in interaction cost. This reduction in trial-and-error operations further translates into lower end-to-end latency. In particular, MobileExplorer completes tasks in 185.82 s on average, reducing the overall latency by 15.9% compared with VLM base model. Importantly, this improvement is achieved without increasing per-step reasoning overhead, since exploration is performed in parallel with model reasoning and utilizes otherwise idle inference time.

We further analyze category-level performance in Fig. ?? and Fig. 8c. MobileExplorer achieves larger gains on exploration-intensive tasks, such as screen reading, complex UI understanding, and information retrieval, where the agent must search for relevant UI information among many candidates. For these tasks, online exploration helps reveal hidden UI branches and reduces redundant trial-and-error steps. In contrast, the benefit is smaller for deterministic editing tasks such as data edit, where the correct action sequence is usually more fixed and exploration may provide less additional information. This confirms that MobileExplorer is particularly effective for search and information-collection scenarios, while editing-oriented tasks may benefit from more conservative exploration.

6.3 Understanding MobileExplorer’s Performance

All experiments in this subsection run VLM inference on the default Jetson AGX Orin platform.

6.3.1 Ablation Study. Fig. 9a and Fig. 9b evaluate the contribution of key components in MobileExplorer. Replacing adaptive speculative branch search with random exploration reduces the success rate from 50.9% to 42.2%, as random probes often waste the limited reasoning window on task-irrelevant UI branches. Removing dual-path state recovery causes the largest degradation, reducing the success rate to 39.7% and increasing both end-to-end latency and average steps, since failed recovery may drift the agent

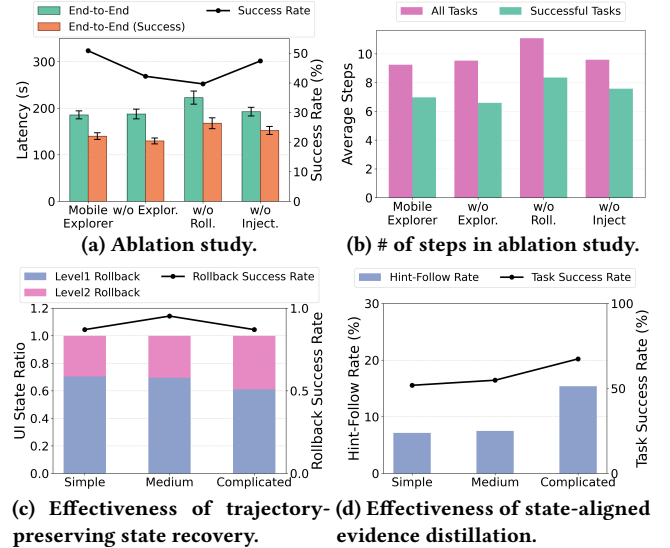


Figure 9: Understanding MobileExplorer’s performance.

away from the intended UI state. Disabling certainty-aware evidence distillation further lowers the success rate to 47.4% and increases the number of steps, because useful exploration results cannot be reliably filtered and reused in later reasoning. Overall, these results show that adaptive exploration, reliable recovery, and evidence injection jointly improve both task success and execution efficiency.

6.3.2 Effectiveness of trajectory-preserving state recovery. We analyze rollback events on UI pages categorized as *Simple*, *Medium*, and *Complicated* based on the number of clickable elements. As shown in Fig. 9c, Level-1 rollback handles most cases, while complicated pages show a higher proportion of Level-2 rollbacks, indicating that simple backtracking is more likely to fail in deeper or less reversible navigation paths. Despite this, the overall rollback success rate remains consistently high across all categories, demonstrating that Level-2 recovery effectively complements Level-1 rollback.

6.3.3 Effectiveness of state-aligned evidence distillation. To evaluate whether exploration-derived evidence contributes to task completion, we measure the evidence-follow rate and relate it to

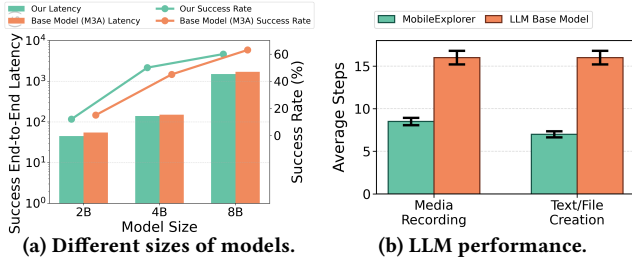


Figure 10: Micro benchmark performance.

task success. We aggregate step-level evidence hits into a task-level metric and group tasks into three complexity levels (*Simple*, *Medium*, and *Complicated*). Fig. 9d shows that the evidence-follow rate increases as UI complexity grows. Meanwhile, tasks with higher evidence-follow rates consistently exhibit higher success rates across all complexity levels. This observation suggests that exploration-derived evidence provides useful guidance, especially when the interface contains many candidate elements.

6.4 Micro Benchmark Performance

Both micro benchmarks in this subsection run VLM inference on the default Jetson AGX Orin platform.

6.4.1 Performance with different model sizes. To further examine the robustness of our design across different model scales, we evaluate the agents using MAI-UI models of different sizes, including 2B and 8B. Fig. 10a reports the end-to-end latency and success rate under these settings. Across different model sizes, MobileExplorer consistently maintains comparable task success rates while reducing the average number of interaction steps, leading to lower end-to-end latency. In particular, our state-aligned evidence distillation typically saves around one reasoning step per task, demonstrating that the benefit of online exploration generalizes across models with different capacities.

6.4.2 Performance on Text-based Agent. MobileExplorer also works with a text-only LLM agent, which improves the success rate from 13.8% to 20.7% with Qwen3-4B[23] as the base model. The gains mainly come from complex tasks that require longer UI exploration and recovery. Since these tasks often need more interactions, the overall latency improvement is limited. MobileExplorer still reduces steps in some cases, such as media recording and text/file creation, where it saves about 7-9 steps over the base model.

6.5 System Overhead

The platform comparison in this subsection is the only experiment that changes the inference device. We evaluate the system overhead of MobileExplorer across a commercial smartphone, Jetson AGX Orin, and a laptop. Fig. 11a reports the end-to-end latency, while Fig. 11b shows the peak memory usage, and Fig. 11c shows the peak power consumption. Across all devices, MobileExplorer consistently reduces end-to-end latency compared with the baseline while maintaining nearly identical memory and power usage. This indicates that the exploration controller introduces only minimal additional system overhead.

We further analyze the runtime overhead of individual modules on the default Jetson AGX Orin platform. Fig. 12 shows that the

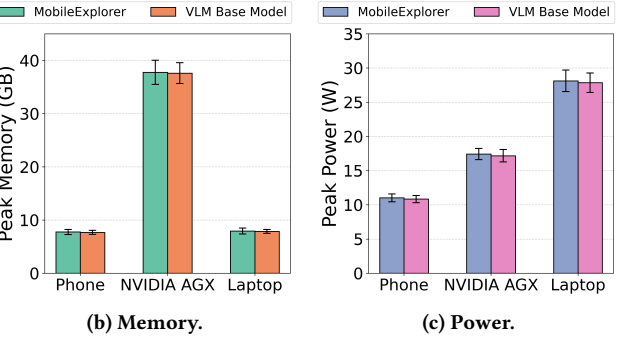
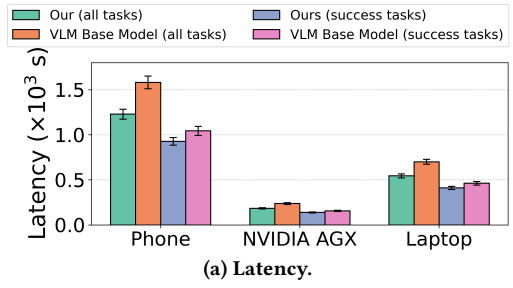


Figure 11: System performance on different platforms.

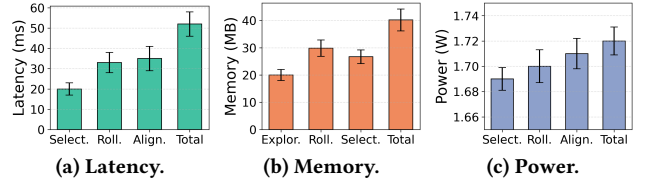


Figure 12: Overhead of different components on the Nvidia AGX Orin.

additional control logic introduces only tens of milliseconds of latency per step, which is negligible compared with multi-second VLM inference. Rollback verification incurs the largest latency due to screenshot-based state checking, while exploration and element selection remain lightweight. Memory and power usage also remain stable across modules.

6.6 Real-world Challenging Case Study

While AndroidWorld provides a controlled benchmark for evaluating mobile GUI agents, its tasks are relatively simplified and do not capture several important characteristics of real mobile environments. In practice, mobile applications contain significantly more complex UI structures, experience dynamic resource conditions, and frequently introduce interrupt-driven UI changes such as system dialogs or notifications. To evaluate how these factors affect agent behavior, we construct a set of real-world smartphone tasks using popular mobile applications and analyze them along three practical dimensions, as shown in Fig. 13. In this case study, VLM inference runs on the default Jetson AGX Orin platform, while the smartphone is used as the GUI execution target for real applications and interruptions.

Complicated UI Elements. Many AndroidWorld tasks are executed in relatively simple applications with limited UI elements

Setting	Feature	Task
Complicated UI Elements	48 elements/page on average (while < 20 in AndroidWorld)	Trip planning in Trip.com with random cities and dynamic attraction lists.
Pop-up Interfering Elements	Interfering events: alarm, message, call, app notifications	Input text in Notes with varying interruptions.
Resource Dynamics	Background tasks: video and music playing	Modify system settings (Bluetooth, WiFi) under dynamic load.

Table 3: Task categories and features in the case study with dynamic variations.

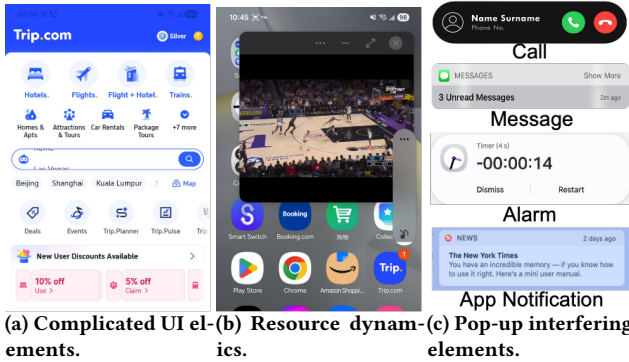


Figure 13: Examples of settings in the case study.

and shallow interface hierarchies. On average, AndroidWorld pages contain about 19.75 interactive elements, whereas real-world mobile applications often present much denser interfaces. For instance, typical screens in the Trip application contain around 48 interactive elements due to long result lists and recommendation panels. To evaluate the agent under such conditions, we design a trip planning task where the agent searches for attractions in the Trip application. We randomly generate 10 different city queries to create diverse UI states, requiring the agent to navigate through dense search results to locate the target item.

Interrupt On-pop Elements. Real mobile environments often introduce unexpected interruptions that temporarily modify the interface structure or occlude interactive elements, such as alarms, message notifications, or incoming calls. These interruptions are largely absent in AndroidWorld, where tasks run in stable environments. To emulate such scenarios, we design a stopwatch task in which different types of interruptions (alarm, message, call, and application notifications) are injected during execution, forcing the agent to correctly recover the UI state before continuing.

Resource Dynamics. Unlike controlled benchmark environments, real smartphones operate under dynamic system conditions where available resources fluctuate due to concurrent background activities, as shown in Fig. 14. Background workloads such as video playback or music streaming may consume CPU and memory resources, affecting the latency of on-device perception and reasoning

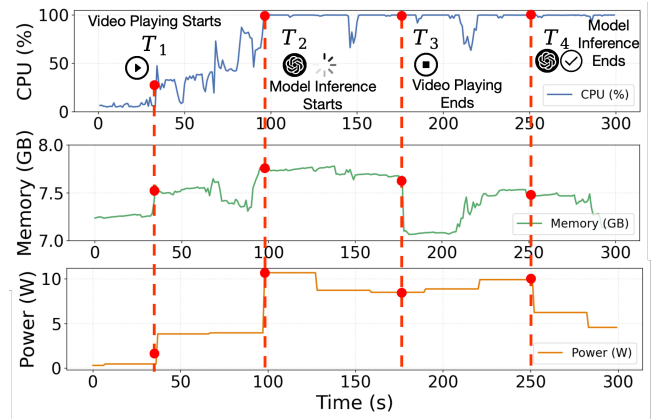


Figure 14: Resource dynamics on mobile phone. T_1 to T_3 : Video playing, T_2 to T_4 : VLM reasoning.

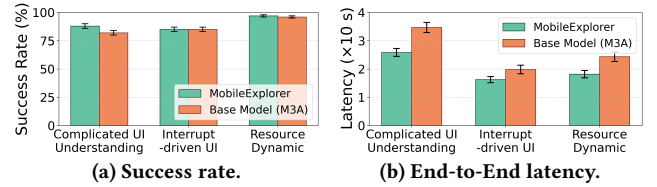


Figure 15: Performance in different complicated settings of the case study.

modules. Since AndroidWorld does not model such resource contention, we design a system settings task where the agent toggles Bluetooth or WiFi while background applications run concurrently, creating realistic resource dynamics during execution.

Results. Together, these tasks evaluate MobileExplorer in realistic mobile scenarios beyond the AndroidWorld benchmark. Each task is repeated three times and the averaged results are shown in Fig. 15. MobileExplorer consistently reduces end-to-end latency while achieving comparable or higher success rates across all settings. The improvement is most significant for complicated UI environments, and the results also demonstrate robustness under interrupt-driven UI changes and dynamic resource conditions.

7 CONCLUSION AND DISCUSSIONS

In this paper, we introduce MobileExplorer, an on-device mobile GUI agent framework that improves task efficiency through intent-aware online exploration. Instead of leaving the system idle during expensive VLM reasoning, MobileExplorer utilizes the reasoning window to probe semantically relevant UI branches and collect additional interface context, while trajectory-preserving state recovery ensures that exploration does not disrupt the main execution trajectory. The discovered observations are further transformed into state-aligned evidence for subsequent reasoning. In future work, we will explore more adaptive exploration strategies that dynamically adjust exploration paths based on task demands and UI complexity, and investigate more complicated interplay between UI exploration and model reasoning to further improve the efficiency of on-device GUI agent systems.

REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. 2025. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923* (2025).
- [3] Jinha Chung, Byeongjun Shin, Jiin Kim, and Minsoo Rhu. 2026. Agent-X: Full Pipeline Acceleration of On-device AI Agents. *arXiv preprint arXiv:2605.10380* (2026).
- [4] Gaole Dai, Shiqi Jiang, Ting Cao, Yuanchun Li, Yuqing Yang, Rui Tan, Mo Li, and Lili Qiu. 2025. Advancing mobile gui agents: A verifier-driven approach to practical deployment. *arXiv preprint arXiv:2503.15937* (2025).
- [5] Tinghe Ding. 2024. Mobileagent: enhancing mobile control via human-machine interaction and sop integration. *arXiv preprint arXiv:2401.04124* (2024).
- [6] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv e-prints* (2024), arXiv-2407.
- [7] Google Android Developers. [n. d.]. Android Debug Bridge (adb). <https://developer.android.com/tools/adb>. ([n. d.]).
- [8] Wenyi Hong, Weihang Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14281–14290.
- [9] Wenyi Hong, Wenmeng Yu, Xiaotao Gu, Guo Wang, Guobing Gan, Haomiao Tang, Jiale Cheng, Ji Qi, Junhui Ji, Lihang Pan, et al. 2025. Glm-4.5 v and glm-4.1 v-thinking: Towards versatile multimodal reasoning with scalable reinforcement learning. *arXiv preprint arXiv:2507.01006* (2025).
- [10] Sunjae Lee, Junyoung Choi, Jungjae Lee, Munim Hasan Wasi, Hojun Choi, Steve Ko, Sangeun Oh, and Insik Shin. 2024. Mobilegpt: Augmenting llm with human-like app memory for mobile task automation. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 1119–1133.
- [11] Wei Li, William E Bishop, Alice Li, Christopher Rawles, Folawiyi Campbell-Ajala, Divya Tyamagundlu, and Oriana Riva. 2024. On the effects of data scale on ui control agents. *Advances in Neural Information Processing Systems* 37 (2024), 92130–92154.
- [12] Kevin Qinghong Lin, Linjie Li, Difei Gao, Zhengyuan Yang, Shiwei Wu, Zechen Bai, Stan Weixian Lei, Lijuan Wang, and Mike Zheng Shou. 2025. Showui: One vision-language-action model for gui visual agent. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 19498–19508.
- [13] Zhibo Liu, Zheng Xu, Zhiqi Zhang, Huina Sun, Jialu Zhang, Qianqian Ye, Jinhui Yuan, Zhi Xu, Lichao Sun, and Li Li. 2026. Mobile GUI Agents under Real-world Threats: Are We There Yet?. In *Proceedings of the 24th Annual International Conference on Mobile Systems, Applications and Services*.
- [14] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, et al. 2024. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573* (2024).
- [15] Christopher Rawles, Alice Li, Daniel Rodriguez, Oriana Riva, and Timothy Lillcrap. [n. d.]. Android in the wild: A large-scale dataset for android device control, 2023. URL <https://arxiv.org/abs/2307.10088> ([n. d.]).
- [16] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [17] Shizuo Tian, Hao Wen, Yuxuan Chen, Jiacheng Liu, Shanhuai Zhao, Guohong Liu, Ju Ren, Yunxin Liu, and Yuanchun Li. 2025. AgentProg: Empowering Long-Horizon GUI Agents with Program-Guided Context Management. *arXiv preprint arXiv:2512.10371* (2025).
- [18] Haoming Wang, Haoyang Zou, Huatong Song, Jiazheng Xu, Junjie Fang, Junting Lu, Longxiang Liu, Qinyu Luo, Shihao Liang, Shijue Huang, et al. 2025. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544* (2025).
- [19] Hao Wen, Yuanchun Li, Guohong Liu, Shanhuai Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 543–557.
- [20] Hao Wen, Shizuo Tian, Borislav Pavlov, Wenjie Du, Yixuan Li, Ge Chang, Shanhuai Zhao, Jiacheng Liu, Yunxin Liu, Ya-Qin Zhang, et al. 2025. Autodroid-v2: Boosting slm-based gui agents via code generation. In *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*. 223–235.
- [21] Bin Xie, Rui Shao, Gongwei Chen, Kaiwen Zhou, Yinchuan Li, Jie Liu, Min Zhang, and Liqiang Nie. 2025. Gui-explorer: Autonomous exploration and mining of transition-aware knowledge for gui agent. *arXiv preprint arXiv:2505.16827* (2025).
- [22] Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. 2025. Step-gui technical report. *arXiv preprint arXiv:2512.15431* (2025).
- [23] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [24] Jiabo Ye, Xi Zhang, Haiyang Xu, Haowei Liu, Junyang Wang, Zhaoqing Zhu, Ziwei Zheng, Feiyu Gao, Junjie Cao, Zhengxi Lu, et al. 2025. Mobile-agent-v3: Fundamental agents for gui automation. *arXiv preprint arXiv:2508.15144* (2025).
- [25] Yao Zhang, Zijian Ma, Yunpu Ma, Zhen Han, Yu Wu, and Volker Tresp. 2025. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 23378–23386.
- [26] Shanhuai Zhao, Hao Wen, Wenjie Du, Cheng Liang, Yunxin Liu, Xiaozhou Ye, Ye Ouyang, and Yuanchun Li. 2025. LLM-Explorer: Towards Efficient and Affordable LLM-based Exploration for Mobile Apps. In *Proceedings of the 31st Annual International Conference on Mobile Computing and Networking*. 589–603.
- [27] Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, et al. 2025. MAI-UI Technical Report: Real-World Centric Foundation GUI Agents. *arXiv preprint arXiv:2512.22047* (2025).